

Source-Level Key-Provenance Analysis

SOURCE ANALYSIS AND TRUSTED VERIFICATION | 25 SEPTEMBER 2026

What is implemented

A bounded Python AST interpreter traces the declared record identifier through assignments, integer arithmetic, aliases and helper calls into protection sinks. It detects finite key pools hidden by modulo/masking and shared matrix blocks assembled through concatenation. It does not need a YAML condition name or execute the analyzed source.

Abstract value	Meaning under the stated contracts
fixed	Same value within the audited record context
record-injective	Distinct integer IDs give distinct idealized keys
bounded(K)	At most K values; reuse possible, not always observed
unknown	Unsupported behavior; never treated as securely fresh

Conditional soundness, not a certificate

An expression-by-expression induction supports fixedness, cardinality and injectivity only for the supported straight-line subset, unique integer IDs, fixed context inputs, reviewed callee contracts and an ideal collision-free KDF. Real research keys are 64-bit truncated hashes. Freshness does not establish key secrecy, independent matrices, unlinkability or production safety.

Coverage and engineering checks

Branches, loops, recursion, dynamic calls and arbitrary numerical correlation are not modeled. The existing branch-heavy experiment runner returns unknown; the three executable study recipes are covered. Module-binding regressions found after execution were fixed; the executed analyzer is archived with its original hash. All 24 benchmark predictions remain unchanged after the fix.

Research status

A domain-specific prototype, not a proven new general analysis method. Abstract interpretation and key-freshness principles are established prior work; specialist priority review remains open.

Proposed Mathematical Construction

SOURCE ANALYSIS AND TRUSTED VERIFICATION | 25 SEPTEMBER 2026

Exposure reuse mass

Let E contain target-training and unordered target-target record pairs. Component j occupies fraction w_j of the protected output; Z_{rj} identifies its realized transform for record r . The proposed functional counts expected weighted repeated components, not leaked identity bits.

$$\mathcal{R}_E = \sum_{(r,s) \in E} \sum_j w_j \Pr[Z_{rj} = Z_{sj}]$$

$$\mathcal{R}_E = (nm + n(n-1)/2) \sum_j w_j / K_j$$

When the closed form applies

The second expression assumes IID uniform slots in pools K_j , n target records and m training records. Ideal fresh components contribute zero; a fixed shared fraction ρ gives $|E| \cdot \rho$. Biased allocations require their actual collision probabilities. A capacity bound alone does not give an upper privacy bound. A separate unweighted collision union bound is derived only for independent hidden isotropic components; it is often vacuous and excludes PolyProtect and the repository's dependent correlated-projection construction.

Utility-constrained acceptance criterion

L is attack top-1, U is legitimate TAR, F is legitimate FMR. A policy change is accepted only if leakage falls, utility is noninferior within a fixed margin, and FMR stays below a ceiling.

$$\begin{aligned} \mathcal{G}_{\delta, \tau} = & \mathbf{1}\{\text{LCB}(L_b - L_c) > 0\} \\ & \times \mathbf{1}\{\text{LCB}(U_c - U_b) \geq -\delta\} \\ & \times \mathbf{1}\{\text{UCB}(F_c) \leq \tau\} \end{aligned}$$

These combine established expectation, coupling and noninferiority ideas; novelty is not certified.

Benchmark and Independent-Review Gap

SOURCE ANALYSIS AND TRUSTED VERIFICATION | 25 SEPTEMBER 2026

24 source cases, two baseline approaches

A separate runtime oracle enumerates component keys for 64 record IDs per case. Source fixtures include constants, affine/helper variants, hidden pools, shared prefixes, explicit matrix assembly and unsupported constructs. All cases were authored in this session: these are development/stress checks, not independently labeled or genuinely held-out evidence.

Method	TP	FP	TN	FN	Abstain
Source interpreter	13	0	7	0	4
YAML checker	0	0	7	17	0
Bandit 1.8.6	1	0	7	16	0

Do not hide abstentions

The source interpreter classifies 20/24 cases (83.3% coverage). It detects 13/17 observed-reuse cases (76.5% when abstentions count as undetected), with four reuse cases left unknown. The seven conditional-fresh cases have no repeated labels in this finite runtime domain; this is not proof over every integer input or a real-world false-positive estimate.

What the baseline comparison means

The YAML baseline receives the same declared fresh-key condition, so it cannot see hidden implementation changes. Bandit is a general Python security scanner, not a biometric-reuse analyzer. Its one warning is B311 for a non-cryptographic RNG, not an inferred transform-reuse finding. This comparison does not establish superiority over specialist static-analysis tools.

External validation still required

Case sources, runtime labels, predictions, raw Bandit JSON and a blank independent-reviewer label column are exported. An external reviewer must supply labels and new constructions without tuning the analyzer to them before journal-level generalization can be claimed.

Leakage Falls; Utility Gate Is Unmet

SOURCE ANALYSIS AND TRUSTED VERIFICATION | 25 SEPTEMBER 2026

Matched pilot: 18 trained endpoints

Completed in 75.64 seconds. MOBIO/FEI; sign-corrected 64-bit BioHash; pool 4, 16/64 shared projection, fresh keys; three joint key/model seeds; one identity partition. The trusted verifier re-encodes each raw probe under the claimed enrollment key. Thresholds use validation identities only; test identities remain held out. Keys are outside the attacker database.

Data	Attack b/c %	TAR b/c %	FMR b/c %
MOBIO	27.08 / 3.47	95.45 / 96.87	0.59 / 0.77
FEI	27.92 / 2.50	95.71 / 95.67	0.91 / 0.80

Data	Leakage drop LCB	TAR change LCB	FMR UCB	Gate
MOBIO	0.42 pp	-3.94 pp	1.43%	FAIL
FEI	5.00 pp	-5.68 pp	1.14%	FAIL

The fixed acceptance criterion fails on utility uncertainty

The TAR lower bound must be at least -3 points. Neither dataset meets it, despite similar or higher average TAR. Both meet positive leakage-reduction and $\leq 2\%$ FMR upper-bound conditions. Six one-sided bounds use Bonferroni tail allocation .05/6 and 10,000 crossed seed/identity bootstrap draws. Margins were fixed before execution and were not relaxed after failure.

Conclusion and costs

Reduced learned leakage is supported here; utility noninferiority is not demonstrated. The architecture requires a trusted raw-probe path and external key custody. It does not defend against key-service compromise or unrestricted compatible-output queries. Uncertainty is conditional on the enrollment gallery; three joint seeds and one partition are limited. No universal privacy, deployment readiness or journal novelty claim follows.

Full protocol, provenance, numerical tables and theory: [experiments/source_security_utility_2026-09-25/](#)